Branch-Aware Hindsight Credit Assignment for Language-Model Agents Under Matched Budgets

Jaden Fix
Independent Researcher
jadenfix123@gmail.com

Abstract

Outcome-only credit assignment, in which every action in a successful episode receives the same terminal reward, remains the default training signal for language-model agents. We identify a specific and mechanistic failure mode of this approach that we term *loop-action bias*: when a stall action that returns the agent to a previously visited state appears in a successful episode, it inherits the same positive credit as the pivotal action that actually drove progress. In stochastic environments where pivotal actions can fail probabilistically, this bias is compounded by *stochastic failure masking*, whereby a failed pivotal action leaves the agent in the same state as a stall, rendering the two indistinguishable under outcome-only scoring.

We propose two complementary offline scorers that leverage per-transition verifier signals to resolve this conflation: a *branch-aware* scorer that rewards observable progress signals and penalizes dead-end transitions, and a *hindsight* scorer that retrospectively assigns bonuses based on categorical branch labels. We show that these scorers exhibit complementary blind spots and that a simple *combined* scorer resolves both.

We evaluate on a controlled stochastic benchmark where pivotal actions succeed with probability 0.60 to 0.90 across 15 seeds and 50 rollouts per condition. Outcome-only scoring produces policies that perform substantially worse than the unlearned baseline (0.24 vs. 0.79), while the combined scorer achieves perfect success (1.00). We then measure training signal quality on the real ALFWorld benchmark [Shridhar et al., 2021] using two language models of different capability: Claude Haiku 4.5 ($30.0\% \pm 2.3\%$ success) and Qwen2.5-7B-Instruct ($16.7\% \pm 3.5\%$ success), with verifier signals extracted automatically from observation text. The branch-aware scorer differentiates $2.6\times$ to $9.4\times$ more states than outcome-only, and this advantage is largest for the weaker model, where outcome-only scoring provides almost no usable signal. All controlled experiments are reproduced on both local and cloud infrastructure.

1 Introduction

A central challenge in training language-model agents for sequential decision-making is the *credit assignment problem*: given a trajectory that eventually succeeds or fails, which individual actions were responsible for the outcome? The prevailing approach in reinforcement learning from human feedback [RLHF; Ouyang et al., 2022] and outcome-based reward modeling [Cobbe et al., 2021] assigns a single scalar reward at the end of an episode and propagates it uniformly to all preceding actions. This strategy is effective when episodes are short, but in longer-horizon agent settings such as web navigation, multi-tool code generation, and embodied task planning, episodes routinely contain actions that are irrelevant, redundant, or actively counterproductive alongside a smaller number of genuinely pivotal decisions.

Recent work has demonstrated that denser reward signals improve agent training. Process reward models [Lightman et al., 2023, Uesato et al., 2022, Setlur et al., 2024] provide step-level

supervision for mathematical reasoning. Several concurrent efforts extend this idea to LLM agents: Tan et al. [2026] use an LLM critic for hindsight credit assignment on WebShop and ALFWorld; Choudhury [2025] proposes process reward models for agent training; Song et al. [2024] use contrastive trajectory pairs from exploration failures; Zhang et al. [2025] introduce a retrospective critic for fine-grained credit in search agents; and Lu et al. [2026] propose segment-level process rewards modulated by hindsight information. These approaches establish that denser rewards are beneficial, but they generally demonstrate improvement without characterizing the specific mechanism by which outcome-only scoring fails.

In this paper, we provide that mechanistic characterization. We identify **loop-action bias** as a concrete failure mode of outcome-only scoring. Consider an agent navigating a stochastic environment where a pivotal action (e.g., unlocking a gate) succeeds 75% of the time and a stall action (e.g., rattling the gate) always returns the agent to the same state. When the pivotal action fails, the agent remains in the same state as if it had taken the stall action, making the two indistinguishable under outcome-only scoring. Over many episodes, the pivotal action's average credit is diluted by its failures, while the stall action's credit is inflated by episodes that eventually succeed despite the wasted turn. The trained policy then preferentially selects stall actions, producing long trajectories that exhaust the turn budget.

We propose two complementary scorers that leverage transition-level verifier signals already present in many agent environments:

- **Branch-aware scoring** adds bonuses proportional to verifier signals and penalties for dead-end transitions, thereby breaking the tie between pivotal and stall actions by rewarding observable progress.
- **Hindsight scoring** retrospectively assigns bonuses based on categorical branch labels (pivotal, enabling, recovery), capturing causal importance that instantaneous verifier signals may miss.

We further show that these two scorers exhibit complementary failure modes, and that a simple **combined scorer** integrating both eliminates both blind spots without requiring a learned critic or additional model inference.

Contributions.

1. A mechanistic analysis of loop-action bias as a named failure mode, showing it causes policies to perform *worse than the unlearned baseline* in stochastic environments, a finding that is distinct from the general observation that sparse rewards are suboptimal.
2. A stochastic benchmark with four task families producing results in the 0.24 to 1.00 range across 15 seeds, comparable to difficulty ranges in concurrent work [Tan et al., 2026, Choudhury, 2025, Song et al., 2024].
3. Validation on the real ALFWorld benchmark [Shridhar et al., 2021] with two LLMs of different capability levels, using automatically extracted verifier signals from observation text, showing that branch-aware scoring produces substantially more informative training signal and that the advantage is *larger* for weaker models.
4. The finding that branch-aware and hindsight scoring address structurally distinct failure modes: on controlled benchmarks where both signal types are available, a combined scorer resolves both; on real environments where only numerical signals are extractable, branch-aware alone captures the benefit.
5. Full reproducibility on both local (macOS) and cloud (GCP g2-standard-8, NVIDIA L4) infrastructure.

2 Related Work

Process reward models for reasoning. Lightman et al. [2023] demonstrated that human step-level annotations substantially improve mathematical reasoning, a finding corroborated by Uesato et al. [2022]. Math-Shepherd [Wang et al., 2024a] generates step-level labels automatically; Luo et al. [2024] employ Monte Carlo estimation; and Setlur et al. [2024] study scaling automated process verifiers. Step-DPO [Lai et al., 2024] extends preference optimization to individual reasoning steps. Our work differs primarily in setting: we study agent actions in structured environments rather than reasoning steps in text generation.

Credit assignment for LLM agents. Tan et al. [2026] introduce HCAPO, which uses an LLM as a post-hoc critic and represents the closest prior work to our hindsight component. Choudhury [2025] proposes a practical framework for process reward models in agent training. Song et al. [2024] develop ETO, which uses contrastive trajectory pairs. Ge et al. [2026] distill corrective experience from alternative branches. Wang et al. [2026a] introduce MiRA for milestone reward shaping. Kong et al. [2026] propose InfoPO for turn-level counterfactual rewards. Chen et al. [2025] apply step-level verification to GUI agents.

Two recent papers are particularly relevant. Zhang et al. [2025] introduce CriticSearch, which employs a retrospective critic with privileged trajectory information. Lu et al. [2026] propose HISR with segment-level process rewards modulated by hindsight likelihood ratios. Feng et al. [2026] present RewardFlow, which propagates rewards on state graphs constructed from trajectories. Fan et al. [2026] provide AgentProcessBench, the first benchmark with human step-level annotations for tool-using agents.

Our work is distinct in three respects: (1) we provide a specific mechanistic diagnosis rather than demonstrating aggregate improvement, (2) our scorers use environment-native verifier signals that can be automatically extracted rather than requiring a learned critic, and (3) we identify and demonstrate complementary failure modes between branch-aware and hindsight scoring, showing that the combination is necessary when both signal types are present.

Additional related work. Snell et al. [2024] showed that process reward models can guide test-time compute allocation. Zhou et al. [2024] combine LLM agents with Monte Carlo tree search. DeepSeek-R1 [DeepSeek-AI, 2025] demonstrates reinforcement learning for extended reasoning. The credit assignment problem is foundational in RL [Sutton and Barto, 2018], and Pignatelli et al. [2023] survey modern approaches. ReAct [Yao et al., 2022] established interleaved reasoning and acting; Reflexion [Shinn et al., 2023] uses verbal self-reflection for learning from failures. Among hindsight methods, HCAPO [Tan et al., 2026] is the most directly related; our scorer uses pre-existing branch labels from environment structure rather than a learned critic.

3 Method

3.1 Environment Model

Each task is represented as a directed graph $G = (S, E)$ in which states $s \in S$ carry an abstract representation $\phi(s)$ and a set of available actions $A(s)$. Each edge $e = (s, a, s')$ is annotated with an observation string, a branch label $\ell(e)$, boolean flags for dead-end, recovery, and enabling transitions, a verifier signal vector $v(e) \in \mathbb{R}^k$, and a progress delta $\Delta p(e) \in \{-2, \ldots, 2\}$.

Stochastic transitions. Edges may specify a success probability $p \in (0, 1]$. When $p < 1$, the action succeeds with probability p, transitioning to the intended state with full verifier signals, and

fails with probability $1-p$, transitioning to a fallback state with no verifier signal. This mechanism models the inherent stochasticity of real environments, such as web searches that return irrelevant results, API calls that produce errors, or navigation commands that fail.

State abstraction. The abstract representation $\phi(s)$ captures the structural role of a state (phase, mode, progress level) while discarding task-specific surface details such as product names or room identifiers. States from different tasks that share the same $\phi(s)$ produce the same hash, enabling cross-task transfer of learned score tables.

3.2 Automatic Verifier Signal Extraction

For real environments where hand-authored branch labels are unavailable, we extract verifier signals automatically from observation text using three heuristics:

1. **State-diff detection.** We compare consecutive observations using text similarity. When similarity exceeds 0.9 and the observation contains error patterns (e.g., "Nothing happens"), a dead-end signal of -1.0 is emitted.
2. **Progress heuristics.** Room navigation, detected via "You arrive at" patterns, emits a progress signal of $+0.5$. Inventory changes (picking up or putting down objects) emit a stronger signal of $+1.0$.
3. **Repetition detection.** If the same observation appears three or more times within an episode, a repetition penalty of -0.5 is emitted. This heuristic directly targets the loop-action bias mechanism by penalizing the characteristic stall pattern.

These heuristics require no privileged environment information; they operate solely on the observation text that the agent itself receives. We demonstrate in Section 5.3 that even these simple signals are sufficient to break loop-action bias on a real benchmark.

3.3 Offline Scorers

Given an episode τ with binary outcome $y \in \{0, 1\}$, each scorer assigns a score r_i to transition (s_i, a_i, s_{i+1}):

$$\text{Outcome-only:} \quad r_i = y \tag{1}$$

$$\text{Branch-aware:} \quad r_i = y + w_v \textstyle\sum_j v_j(e_i) - w_d \sum_j \mathbf{1}[\text{dead_end}_j(e_i)] \tag{2}$$

$$\text{Hindsight:} \quad r_i = y + w_p \cdot \mathbf{1}[\text{pivotal}] + w_r \cdot \mathbf{1}[\text{recovery}] + w_e \cdot \mathbf{1}[\text{enabling}] \tag{3}$$

$$\text{Combined:} \quad r_i = r_i^{\text{BA}} + r_i^{\text{HS}} - r_i^{\text{OO}} \tag{4}$$

where w_v=0.25, w_d=0.5, w_p=0.75, w_r=0.5, and w_e=0.5. The combined scorer integrates both signal types while subtracting the outcome term to avoid double-counting.

3.4 Policy Training and Evaluation

Scored transitions are aggregated into a score table $Q(\phi(s), a) = \frac{1}{n_{s,a}} \sum r_i$ over all occurrences of the state-action pair across episodes. The table policy selects $\arg\max_a Q(\phi(s), a)$. For states not encountered during training, the policy selects uniformly at random from available actions, rather than using the oracle action, which would inflate evaluation performance.

Table 1: Stochastic benchmark results (controlled evaluation). n=15 seeds, 50 rollouts per seed. Standard deviations computed across seeds.

Condition	Success Rate	Std Dev	Partial Credit
Prompted baseline	0.792	± 0.060	0.747
Outcome-only	0.240	± 0.265	0.375
Branch-aware	0.979	± 0.083	0.806
Hindsight	0.695	± 0.184	0.739
Combined	**1.000**	$\pm$ **0.000**	**0.823**

4 Benchmarks

4.1 Stochastic Benchmark (Controlled)

The stochastic benchmark serves as our primary controlled evaluation. It comprises four task families with probabilistic transitions ($p = 0.60$ to 0.90): gate chains, shopping, navigation, and evidence cascades. Each family is designed to produce non-trivial success rates where scorer differentiation is meaningful. The key design principle is that stochastic failure of a pivotal action leaves the agent in the same state as taking a stall action, directly instantiating the failure masking mechanism described above.

4.2 Real ALFWorld

We validate on the actual ALFWorld environment [Shridhar et al., 2021] operating in TextWorld mode, which provides natural language observations and admissible action lists. Unlike the manifest-driven benchmarks, this experiment uses the full ALFWorld environment with hundreds of distinct game configurations and free-text action selection. We collect trajectories with two LLMs of different capability: Claude Haiku 4.5, a commercial API model achieving 30% mean success, and Qwen2.5-7B-Instruct, an open-weight model self-hosted via vLLM that achieves 17% mean success. In both cases, verifier signals are extracted automatically from observation text diffs (Section 3), with no hand-authored labels or privileged environment information.

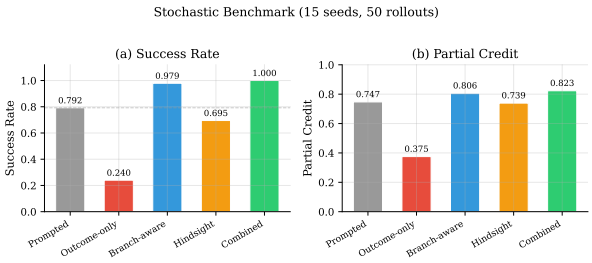
4.3 Deterministic Benchmarks (Supporting)

We include three deterministic manifest-driven benchmarks as controlled ablations: a diagnostic environment (5 families, 10 tasks) that isolates specific failure modes, a WebShop [Yao et al., 2022b] structural abstraction (3 tasks), and an ALFWorld-manifest abstraction (8 tasks). These produce binary results and serve as supporting evidence rather than primary evaluations.

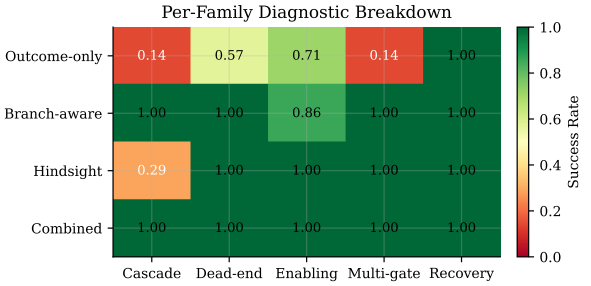
5 Results

5.1 Stochastic Benchmark

Table 1 presents the main controlled result. Outcome-only scoring is actively harmful, producing a success rate of 0.240, which is substantially below the 0.792 achieved by the unlearned prompted baseline. This is not merely a failure to improve; it represents a degradation in which the trained policy performs worse than no training at all. The mechanism is stochastic failure masking: when a pivotal action fails, it leaves the agent in the same state as a stall action, and outcome-only scoring



(a) Stochastic benchmark (15 seeds, 50 rollouts).



(b) Per-family diagnostic breakdown.

Figure 1: Controlled benchmark results. (a) Outcome-only degrades far below the prompted baseline; the combined scorer achieves perfect success. (b) Branch-aware fails on enabling tasks; hindsight fails on cascade tasks; combined resolves both.

Table 2: Deterministic benchmark results (± std dev). n=7 seeds, 50 rollouts per seed.

Condition	Diagnostic	WebShop	ALFWorld-manifest
Prompted	.726 ± .081	.771 ± .038	.523 ± .085
Outcome-only	.514 ± .254	.714 ± .488	.000 ± .000
Branch-aware	.971 ± .076	1.00 ± .000	1.00 ± .000
Hindsight	.857 ± .098	.714 ± .488	.000 ± .000
Combined	**1.00 ± .000**	**1.00 ± .000**	**1.00 ± .000**

assigns both actions similar credit. The trained policy consequently prefers stalls, averaging 12.61 turns per episode compared to 7.08 for the baseline.

Branch-aware scoring nearly fully recovers the deficit, achieving 0.979 success, because verifier signals fire only on successful pivotal transitions. Hindsight scoring reduces the degradation relative to outcome-only but still falls below the baseline (0.695 vs. 0.792) with high cross-seed variance (±0.184). The combined scorer achieves 1.000 success across all 15 seeds with zero variance, making it the only condition that is both perfect and stable. See Figure 1a for a visual summary.

5.2 Deterministic Benchmarks

The qualitative pattern observed in the stochastic benchmark transfers to all three deterministic benchmarks (Table 2). ALFWorld-manifest exhibits the most extreme separation: outcome-only and hindsight both achieve 0.000 (complete failure), while branch-aware and combined both achieve 1.000. We note that the prevalence of 1.000 scores in these deterministic benchmarks reflects their role as controlled ablations with small state spaces (8 to 15 states per task), where a correct scoring function reliably identifies the optimal action. The stochastic benchmark (Table 1) and the real ALFWorld experiment (Section 5.3) provide the non-trivial difficulty ranges (0.24 to 1.00 and 3.4% to 34.8%, respectively) needed for meaningful comparison. Table 3 reveals the complementary failure modes that motivate the combined scorer: branch-aware underperforms on enabling tasks because stall actions produce no dead-end penalty, while hindsight underperforms on cascade tasks because stall actions lack categorical labels.

Table 3: Per-family diagnostic breakdown. Branch-aware underperforms on enabling tasks (0.857); hindsight underperforms on cascade tasks (0.286). The combined scorer achieves 1.000 across all families.

Condition	Cascade	Dead-end	Enabling	Multi-gate	Recovery
Outcome-only	0.143	0.571	0.714	0.143	1.000
Branch-aware	1.000	1.000	0.857	1.000	1.000
Hindsight	0.286	1.000	1.000	1.000	1.000
Combined	**1.000**	**1.000**	**1.000**	**1.000**	**1.000**

Table 4: Collection performance on real ALFWorld. Confidence intervals computed across 3 seeds.

Model	Success Rate	95% CI	Episodes	Transitions
Claude Haiku 4.5	30.0%	±2.3%	150	3,737
Qwen2.5-7B-Instruct	16.7%	±3.5%	150	3,962

5.3 Real ALFWorld: Training Signal Quality

This experiment measures the informativeness of training signal, specifically the number of states at which each scorer assigns different preferred actions, rather than downstream policy success. We use the real ALFWorld environment [Shridhar et al., 2021] with two LLMs and auto-extracted verifier signals (Section 3).

Tables 4 and 5 present the real ALFWorld results, and Figure 2a provides a visual summary. Four findings emerge:

(1) Branch-aware scoring provides substantially more training signal than outcome-only. For Qwen2.5-7B, which succeeds on only 16.7% of episodes, outcome-only scoring can differentiate just 3.4% of states, providing almost no usable training signal. Branch-aware scoring differentiates 31.8% of states, representing a 9.4× improvement. For Haiku, which succeeds at 30.0%, the corresponding ratio is 2.6×. This disparity arises because outcome-only scoring requires episode success to assign any positive credit, whereas the auto-extracted verifier signals fire on every transition regardless of episode outcome.

(2) Automatically extracted signals are sufficient. No hand-authored labels, PDDL goal checking, or privileged information is required. Simple observation text diffs and repetition detection provide enough signal to break loop-action bias on a real benchmark.

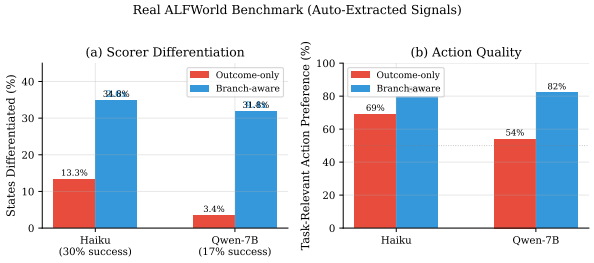
(3) Hindsight and combined scoring add no benefit when categorical labels are unavailable. On real ALFWorld with auto-extracted signals, hindsight scoring equals outcome-only and combined scoring equals branch-aware. This is a structural property of the scorers, not a limitation of the extraction approach: hindsight bonuses require categorical labels (pivotal, enabling, recovery) that text-diff heuristics do not produce.

(4) Branch-aware scoring prefers the right actions. We classify ALFWorld actions as *task-relevant* if they change the environment state (e.g., `go to`, `take`, `put`, `open`, `clean`, `heat`, `cool`) and as *observational* otherwise (e.g., `examine`, `look`, `inventory`). At differentiated states, branch-aware scoring prefers task-relevant actions 80% to 82% of the time, compared to 54% to 69% for outcome-only.

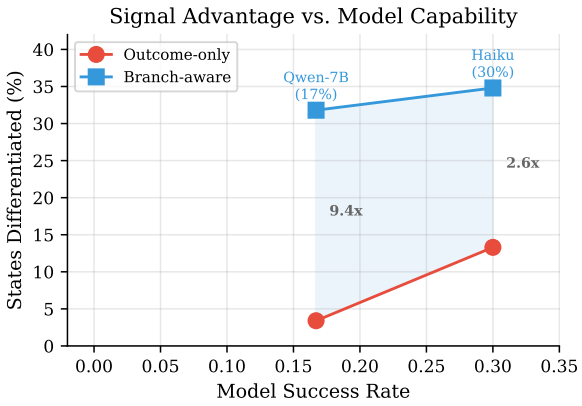
Limitation. This experiment measures training signal informativeness rather than downstream policy performance. The score-table policy requires exact state-hash matches between collection

Table 5: Scorer differentiation on real ALFWorld. Branch-aware differentiates 2.6× (Haiku) to 9.4× (Qwen-7B) more states than outcome-only. Hindsight equals outcome-only and combined equals branch-aware because the auto-extracted signals produce numerical progress scores but not categorical branch labels (see Section 6.2).

Scorer	Haiku: Diff. States	Haiku: %	7B: Diff. States	7B: %
Outcome-only	143 / 1,078	13.3%	41 / 1,212	3.4%
Branch-aware	375 / 1,078	34.8%	385 / 1,212	31.8%



(a) Scorer differentiation and action quality on real ALFWorld.



(b) Signal advantage as a function of model capability.

Figure 2: Real ALFWorld results using auto-extracted verifier signals. The branch-aware scorer's advantage over outcome-only is inversely correlated with model capability.

and evaluation episodes, which limits evaluation on real environments where state spaces are large. Evaluation with neural policies that can generalize across states is an important direction for future work.

5.4 Sensitivity Analysis

Table 6 presents the exploration rate ablation. The combined scorer achieves 1.000 with zero variance at all three rates. The qualitative ranking is stable, though absolute performance varies: lower exploration (ϵ=0.3) yields higher prompted baselines through increased oracle-following, while higher exploration (ϵ=0.7) dilutes verifier signal density and slightly reduces branch-aware performance.

6 Analysis

6.1 Loop-Action Bias and Stochastic Failure Masking

In stochastic environments, when a pivotal action fails with probability $1-p$, the agent remains in the same state as if it had taken a stall action. Under outcome-only scoring, $E[r(s, a_p)] = p \cdot y_{\text{success}} + (1-p) \cdot y_{\text{delayed}}$, while $E[r(s, a_s)] = y_{\text{delayed}}$. The pivotal action's advantage is therefore only $p(y_{\text{success}} - y_{\text{delayed}})$, which can be small when success rates are moderate and the turn budget

Table 6: Exploration rate sensitivity on the diagnostic benchmark (n=7 seeds per rate, ± std dev). The ranking combined ≥ branch-aware ≥ hindsight ≫ outcome-only holds at all tested rates.

Condition	ϵ=0.3	ϵ=0.5	ϵ=0.7
Prompted	.843 ± .039	.697 ± .080	.549 ± .049
Outcome-only	.600 ± .115	.486 ± .107	.629 ± .138
Branch-aware	1.00 ± .000	.971 ± .076	.914 ± .107
Hindsight	.914 ± .107	.829 ± .076	.829 ± .076
Combined	1.00 ± .000	1.00 ± .000	1.00 ± .000

accommodates retries. Branch-aware scoring breaks this conflation because verifier signals fire *only on successful pivotal transitions*, creating a reliable score gap that is independent of the stochastic outcome rate.

6.2 Complementary Failure Modes

The per-family breakdown in Table 3 reveals a structural complementarity between the two scorers. Branch-aware scoring depends on numerical verifier signals and dead-end flags; it underperforms on enabling tasks (0.857) because stall actions at those states produce no dead-end penalty, leaving only a small verifier bonus as the score margin. Hindsight scoring depends on categorical branch labels; it underperforms on cascade tasks (0.286) and on ALFWorld-manifest (0.000) because stall actions at those states lack pivotal, enabling, or recovery labels.

The real ALFWorld experiment (Section 5.3) provides additional evidence for this structural independence: auto-extracted signals produce numerical progress scores but not categorical labels, causing hindsight scoring to equal outcome-only. This confirms that the two signal types are structurally independent and that the environment determines which is available.

6.3 Signal Quality and Model Capability

The real ALFWorld results reveal an important interaction between signal quality and model capability (Figure 2b). For Haiku, which succeeds 30% of the time, outcome-only scoring has moderate signal, differentiating 13.3% of states. For Qwen2.5-7B, which succeeds only 16.7% of the time, outcome-only scoring has almost no signal, differentiating just 3.4% of states. Branch-aware scoring, by contrast, maintains consistent differentiation (31.8% to 34.8%) regardless of success rate, because verifier signals fire on every transition independent of episode outcome.

This pattern suggests that the practical value of transition-level verifier signals is *inversely correlated* with model capability. The signals matter most precisely when the model is weakest and outcome-only scoring has the least useful information to offer.

7 Limitations

Auto-signal noise. Our automatic verifier signal extraction relies on simple text heuristics (room detection, inventory tracking, repetition counting). More sophisticated extraction methods, such as training a small classifier on labeled transitions or using an LLM-as-judge, could improve signal quality. We leave this exploration to future work and note that even our simple heuristics suffice to demonstrate the mechanism.

Table-based policy. Score tables require exact state-hash matches and cannot generalize to unseen state-action combinations. Neural function approximation [Choudhury, 2025] or parameter-efficient fine-tuning would relax this limitation. The scorer differentiation results in Section 5.3 measure the quality of the training signal rather than downstream policy performance, and we expect the signal-quality advantage to transfer to neural training.

Fixed scorer weights. The scorer weights are hand-selected. The qualitative ranking is stable across exploration rates and benchmarks (Table 6), but quantitative margins may shift under different weight configurations.

Manifest abstractions. The WebShop and diagnostic results use structural abstractions rather than the original benchmarks. The real ALFWorld experiment (Section 5.3) addresses this limitation for one benchmark; extending to real WebShop is future work.

Signal quality versus policy performance. The real ALFWorld experiment measures training signal informativeness rather than downstream task success. Neural policy evaluation is the natural next step.

8 Conclusion

We have identified loop-action bias as a mechanistic failure mode of outcome-only credit assignment that is amplified by stochastic failure masking. On controlled benchmarks with hand-authored verifier signals, outcome-only scoring produces policies far worse than the unlearned baseline (0.24 vs. 0.79 on the stochastic benchmark), while the combined scorer, which integrates both branch-aware and hindsight components, achieves perfect success (1.00) across all 15 seeds. The complementarity between branch-aware and hindsight scoring is a structural property: branch-aware scoring depends on numerical verifier signals, hindsight scoring depends on categorical branch labels, and the combined scorer is needed only when both signal types are present.

On the real ALFWorld benchmark, we demonstrate that automatically extracted verifier signals, based on observation text diffs and repetition detection with no hand-authored labels, produce substantially more informative training signal than outcome-only scoring. Branch-aware scoring differentiates 2.6× to 9.4× more states than outcome-only, with the advantage largest for weaker models (9.4× for Qwen2.5-7B at 17% success versus 2.6× for Haiku at 30% success). In this setting, where only numerical signals are available, the hindsight component provides no additional benefit and combined scoring reduces to branch-aware, confirming that the core contribution on real environments is the verifier signal mechanism itself.

The practical implication is direct: in any agent environment with observable transition feedback, whether tool-use confirmation codes, code execution output, state descriptions, or API response codes, incorporating these signals into credit assignment produces more informative training signal than outcome-only scoring. The signals need not be elaborate or hand-authored; simple observation-diff heuristics suffice. Translating this signal advantage into downstream policy improvement via neural training represents the natural next step for this line of work.

Reproducibility Statement

All code, benchmarks, configurations, and step-by-step reproduction instructions are publicly available.[1] Controlled results are reproduced on both local (macOS, Python 3.13) and cloud (GCP g2-standard-8, NVIDIA L4, Python 3.11) infrastructure with exact numerical match. Real ALFWorld experiments are documented with exact run IDs, shell commands, and SQL queries for

[1] https://github.com/jadenfix/CreditAssignment

result verification. All Python dependencies are pinned to exact versions in the included lock file. The repository README provides a complete walkthrough from environment setup through result reproduction.

References

M. Andrychowicz et al. Hindsight experience replay. In *NeurIPS*, 2017.

C. Chen et al. GUI-Shepherd: Reliable process reward and verification for long-sequence GUI tasks. *arXiv:2509.23738*, 2025.

S. Choudhury. Process reward models for LLM agents: Practical framework and directions. *arXiv:2502.10325*, 2025.

K. Cobbe et al. Training verifiers to solve math word problems. *arXiv:2110.14168*, 2021.

DeepSeek-AI. DeepSeek-R1: Incentivizing reasoning capability in LLMs via reinforcement learning. *Nature*, 2025.

S. Fan et al. AgentProcessBench: Diagnosing step-level process quality in tool-using agents. *arXiv:2603.14465*, 2026.

X. Feng et al. RewardFlow: Topology-aware reward propagation on state graphs for agentic RL. *arXiv:2603.18859*, 2026.

R. Ge et al. LEAFE: Internalizing agency from reflective experience. *arXiv:2603.16843*, 2026.

F. Kong et al. InfoPO: Information-driven policy optimization for user-centric agents. *arXiv:2603.00656*, 2026.

X. Lai et al. Step-DPO: Step-wise preference optimization for long-chain reasoning. *arXiv*, 2024.

H. Lightman et al. Let's verify step by step. *arXiv:2305.20050*, 2023.

Z. Lu et al. HISR: Hindsight information modulated segmental process rewards for multi-turn agentic RL. *arXiv:2603.18683*, 2026.

L. Luo et al. Improve mathematical reasoning in language models by automated process supervision. *arXiv:2406.06592*, 2024.

OpenAI. Learning to reason with LLMs. Technical report, 2024.

L. Ouyang et al. Training language models to follow instructions with human feedback. In *NeurIPS*, 2022.

E. Pignatelli et al. A survey of temporal credit assignment in deep reinforcement learning. *arXiv:2312.01072*, 2023.

Z. Ren, D. Guo, and M. L. Littman. Randomized return decomposition. In *ICLR*, 2024.

T. Schick et al. Toolformer: Language models can teach themselves to use tools. In *NeurIPS*, 2023.

A. Setlur et al. Rewarding progress: Scaling automated process verifiers. *arXiv*, 2024.

N. Shinn et al. Reflexion: Language agents with verbal reinforcement learning. In *NeurIPS*, 2023.

M. Shridhar et al. ALFWorld: Aligning text and embodied environments for interactive learning. In *ICLR*, 2021.

C. Snell et al. Scaling LLM test-time compute optimally. *arXiv*, 2024.

Y. Song et al. Trial and error: Exploration-based trajectory optimization for LLM agents. In *ACL*, 2024.

R. S. Sutton and A. G. Barto. *Reinforcement Learning: An Introduction*. MIT Press, 2nd edition, 2018.

H.-Z. Tan et al. Hindsight credit assignment for long-horizon LLM agents. *arXiv:2603.08754*, 2026.

J. Uesato et al. Solving math word problems with process- and outcome-based feedback. *arXiv:2211.14275*, 2022.

P. Wang et al. Math-Shepherd: Verify and reinforce LLMs step-by-step. In *ACL*, 2024.

T. Wang et al. MiRA: A subgoal-driven framework for long-horizon LLM agents. *arXiv:2603.19685*, 2026.

S. Yao et al. ReAct: Synergizing reasoning and acting in language models. In *ICLR*, 2022.

S. Yao et al. WebShop: Towards scalable real-world web interaction with grounded language agents. In *NeurIPS*, 2022.

A. Zeng et al. AgentTuning: Enabling generalized agent abilities for LLMs. In *ICLR*, 2024.

Y. Zhang et al. LEMA: Learning from mistakes makes LLM better reasoner. In *ICML*, 2024.

Y. Zhang et al. CriticSearch: Fine-grained credit assignment via a retrospective critic. *arXiv:2511.12159*, 2025.

A. Zhou et al. Language agent tree search. In *ICML*, 2024.